TP 3

-

Outils DevOps

GIT & CI-CD

Date	24/02/2026
Etudiant	44003786

Table des matières

Outils DevOps **1**
I. Introduction à Git 3
Étape 1 – Initialisation d'un dépôt Git 3
Questions : 3
Étape 2 – Initialisation d'un dépôt Git 4
Questions : 5
Étape 3 – Création d'une branche et fusion 6
Questions : 7
Étape 4 – Pousser le code sur GitLab 8
Questions : 13
II. CI/CD 14
Étape 1 – Création de l'application 15
Étape 2 – Déploiement et vérification manuel 15
Questions : 15
Étape 3 – En autonomie : Déploiement et vérification automatique 16
1. Installation GitLab Runner 16
Questions : 20
III. Conclusions 22
IV. Annexe 22
Contenu fichiers .gitlab-ci.yml : 22

I. Introduction à Git

Git est un système de **versioning** (gestion de versions).
Git permet notamment de garder un **historique** de chaque **modification**, travailler à plusieurs sur le même projet ou encore tester des nouveautés sans casser la version qui fonctionne.

Étape 1 – Initialisation d'un dépôt Git

Installer Git sur la VM :

```
$apt update
$apt install git-all
```

Créer un répertoire et initier Git :

```
$mkdir tp-git && cd tp-git
$git init
```

Vérification création dépôt Git :

```
root@debian:~/tp-git# ls -la
total 12
drwxrwxr-x 3 root root 4096 24 févr. 08:36 .
drwx------ 5 root root 4096 24 févr. 08:36 ..
drwxrwxr-x 7 root root 4096 24 févr. 08:36 .git
root@debian:~/tp-git#
```

Ici on voit bien que le fichier .git à été créer, donc le dépôt Git à bien été créer.

Questions :

1. À quoi sert la commande git init ? Que contient le dossier. git/ ?
La commande **git init** permet de transformer un dossier en **dépôt Git**.
Le dossier .git est un dossier caché qui est créé lors du git init. Ce dernier permet de réaliser le **versionning** puisqu'il contient toutes les versions du dossier.
Si ce dossier venait à être supprimé on perdrait toutes les **versions** de notre projet.

2. Comment savoir si un dossier est déjà un dépôt Git ?
On peut simplement vérifier si le **fichier caché** .git est présent dans le dossier ou sinon taper la commande :
`$Git status` -> Pour voir le **statut** de git dans le dossier.

Étape 2 – Initialisation d'un dépôt Git

Crée un fichier **README.md** et ajoute une description :

```
$echo "# TP DevOps - HOARAU Thomas" > README.md
```

Vérifie **l'état** du dépôt :

```
$git status
```

```
root@debian:~/tp-git# cat README.md
# TP DevOps - Ce dossier est un dépôt Git - HOARAU Thomas
root@debian:~/tp-git# git status
Sur la branche master

Aucun commit

Fichiers non suivis:
  (utilisez "git add <fichier>..." pour inclure dans ce qui sera validé)
        README.md

aucune modification ajoutée à la validation mais des fichiers non suivis sont pr
ésents (utilisez "git add" pour les suivre)
```

Quand je **vérifie** le **statut** ici on m'indique que pour inclure les **modifications** dans le **dépôt Git** il faut que je fasse un Git add du fichier.

J'utilise la commande :

```
$git add README.md
```

Cette commande me permet **d'ajouter** la modification au **dépôt git**, et donc de ce fait de **sauvegarder** la nouvelle version du dossier.

Pour décrire la nouvelle version du dossier on peut utiliser

```
$git commit -m "Description"
```

Cette commande permet d'ajouter une description à chaque nouvelle version du projet.

Néanmoins, avant d'utiliser le git commit on doit s'identifier auprès de Git afin que les modifications apportées soit au nom de la personne ayant effectué les modifications. Pour cela je lance les commandes suivantes :

```
$git config --global user.email "t.hoarau@rt-iut.re"
$git config --global user.name "HOARAU Thomas"
```

Et je lance ensuite :

```
$git commit -m "Ajout du fichier README"
```

Pour vérifier le commit on peut regarder les **logs** de commit de **git** :

```
$git log -oneline
```

```
root@debian:~/tp-git# git log --oneline
575801a (HEAD -> master) Ajout du fichier README
root@debian:~/tp-git#
```

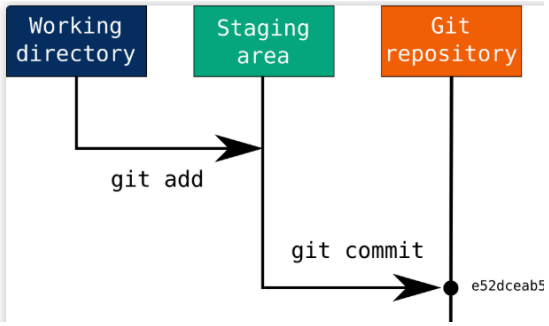
L'historique des **commit** retourne bien le commit qui a été lancer décrivant l'ajout du fichier README.md

Questions :

3. Quelle est la différence entre git add et git commit ?

Git add : Ajoute les modification à la **staging area**, cette zone est une zone **intermédiaire** entre le **working directory** et le **repository** (Dépôt Git)

Git commit : Valide les **modifications** en les déplaçant de la staging area au **dépôt Git** en y ajoutant une **description** des modifications qui ont été apportées.



4. Que fait la commande git log ?

Cette commande permet de **consulter l'historique** des commit qui ont été effectués dans le **dépôt Git**.

5. Que se passe-t-il si on modifie un fichier après l'avoir ajouté mais avant de commit ?

Dans ce cas seule la version du fichier **au moment du git add** sera **enregistrée** par le commit.

En effet, les **modifications** apportées **après l'ajout** ne sont pas enregistrées au niveau de la **staging area** donc le commit ne peux pas les ajouter au **dépôt Git**.

Étape 3 – Création d'une branche et fusion

Créer une branche feature-api :

```
$git branch feature-api
```

Change de branche :

```
$git checkout feature-api
```

Modifie le fichier README.md :

```
$echo "Ajout d'une branche feature-api" >> README.md
```

Commit la modification :

```
git add README.md git commit -m "Mise à jour du README dans feature-api"
```

Reviens sur master et fusionne :

```
$git checkout master
$git merge feature-api
```

```
root@debian:~/tp-git# git checkout master
Basculement sur la branche 'master'
root@debian:~/tp-git# git merge feature-api
Mise à jour 575801a..3ad465a
Fast-forward
 README.md | 1 +
 1 file changed, 1 insertion(+)
root@debian:~/tp-git#
```

Vérifie l'historique des commits :

```
$git log --oneline --graph
```

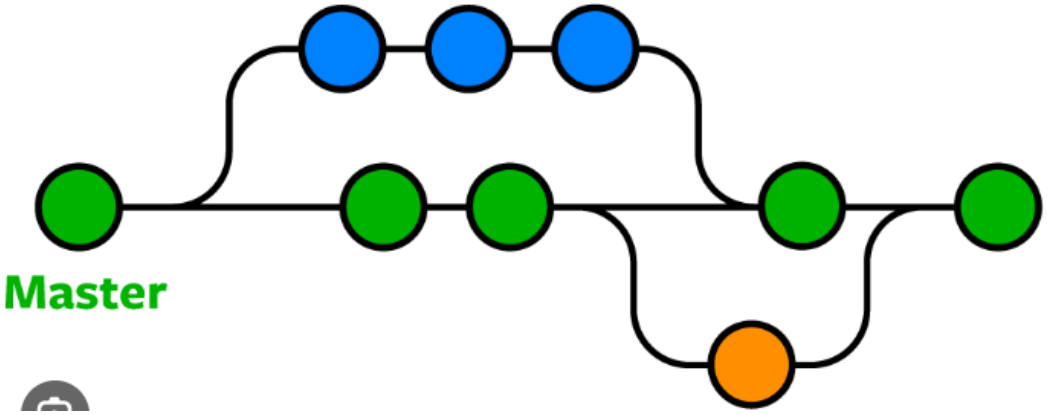
```
root@debian:~/tp-git# git log --oneline --graph
* 3ad465a (HEAD -> master, feature-api) Mise à jour du README dans feature-api
* 575801a Ajout du fichier README
root@debian:~/tp-git#
```

Ici on constate que la **modification** à bien apporté.

Questions :

6. À quoi sert une branche dans Git ?

Une **branche git** est une **ligne de développement séparée**.
Les **branches** permettent de travailler sur une nouvelle fonctionnalité ou la correction d'un bug de façon **parallèle** sans modifier le code **principal**.



7. Que fait la commande git merge ?

La commande **Git merge** permet de **fusionner** des **branches** entre elles.
De ce fait, les branches fusionnent leurs **modifications**.

8. Quelle commande permet de voir les branches existantes ?

La commande est :

```
$git branch
```

```
root@debian:~/tp-git# git branch
  feature-api
* master
root@debian:~/tp-git#
```

Comme on peut le constater, la **branche** sur laquelle on se trouve est indiquée en **vert**.

9. Que se passe-t-il si deux branches ont modifié le même fichier avant la fusion ?

Dans ce cas, cela crée un **conflit de fusion** (merge conflict).
En effet Git ne décide pas quelles **versions** du fichier choisir, donc cela crée un conflit.
Pour régler ce problème, c'est au **développeur** d'ouvrir le fichier et de **choisir** le bon code.

Étape 4 – Pousser le code sur GitLab

GitLab est une plateforme web de gestion de dépôts Git. Elle permet d'héberger du code, de gérer les **versions** et d'intégrer des outils de **collaboration** et **d'automatisation** (CI/CD).

Il faut donc installer GitLab sur la VM, pour cela je vais utiliser un container GitLab. Je vais donc premièrement installer docker pour ensuite pouvoir récupérer une image de gitlab
Installation Docker :

```
$apt install docker.io docker-compose -y
```

Vérification Docker :

```
$systemctl status docker
```

```
root@debian:~/tp-git# systemctl status docker
● docker.service - Docker Application Container Engine
     Loaded: loaded (/usr/lib/systemd/system/docker.serv
     Active: active (running) since Tue 2026-02-24 09:51
 Invocation: 2feac78ad042454a9d717c611909a30a
TriggeredBy: ● docker.socket
```

Installation GitLab via Docker :

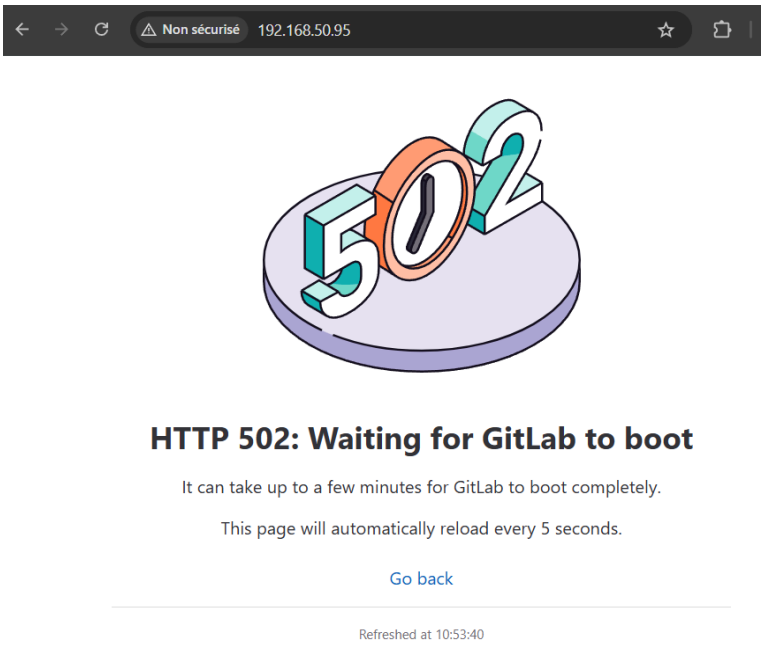
```
$export GITLAB_HOME=$HOME/gitlab
$mkdir -p $GITLAB_HOME
```

Lancement du Docker :

```
$sudo docker run --detach \
  --hostname 192.168.50.95 \
  --publish 443:443 --publish 80:80 --publish 2222:22 \
  --name gitlab \
  --restart always \
  --volume /srv/gitlab/config:/etc/gitlab \
  --volume /srv/gitlab/logs:/var/log/gitlab \
  --volume /srv/gitlab/data:/var/opt/gitlab \
  --shm-size 256m \
  gitlab/gitlab-ce:latest
```

Ici docker récupère l'image Gitlab qui pèse plusieurs Go.

J'ouvre ensuite mon **navigateur** et je tape **l'IP** de ma VM en url pour vérifier que le **container Gitlab** tourne et qu'il soit accessible :



Je constate que le **gitlab** est bien accessible mais que ce dernier est encore en train de démarrer.
En effet, une fois qu'on à lancer le container il faut attendre plusieurs **minutes** que ce dernier se **lance**.

En attendant je peux consulter les **logs** du **démarrage** du container avec :

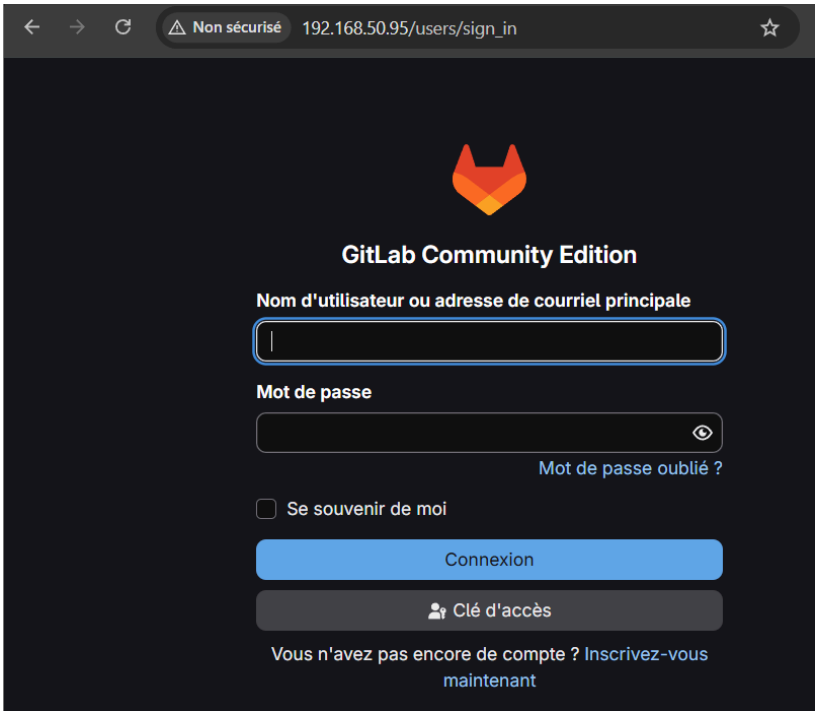
```
$docker logs -f gitlab
```

Et je peux également **récupérer** le **mot de passe root** pour me connecter à **l'interface GitLab** avec la commande :

```
$docker exec -it gitlab grep 'Password:' /etc/gitlab/initial_root_password
```

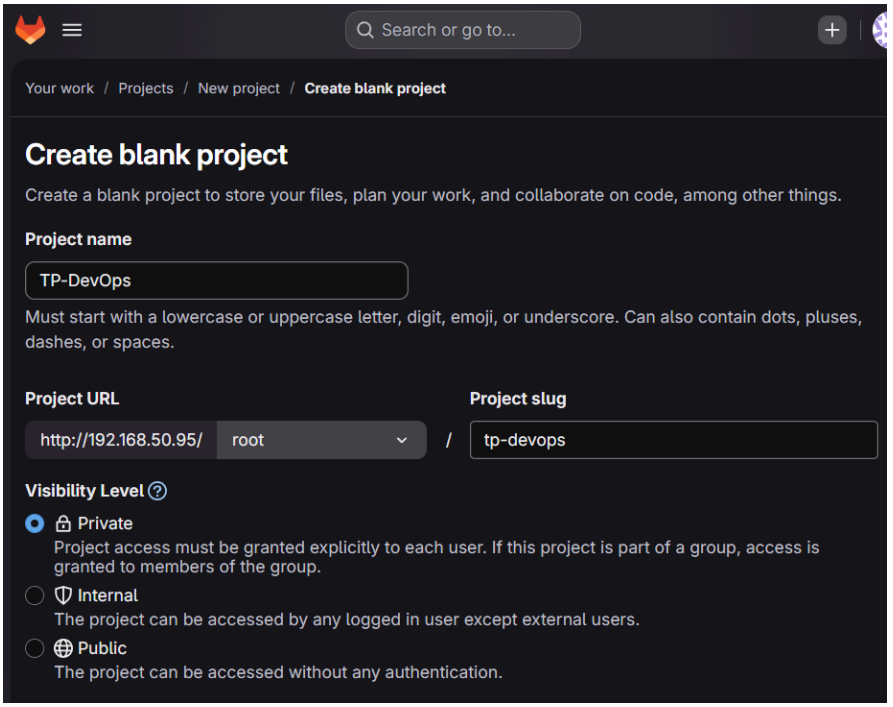
Password : N+fnBDH5okRDmh2VfNmj2JXlWiF438ETdW9GYadARY0=

Une fois le container démarré, voilà la **page** de **connexion** :



Je peux donc me **connecter** avec l'ID root et le mot de passe précédemment récupéré.

Une fois connecté je peux créer un nouveau projet "*TP-DevOps*"



Je peux ensuite **récupérer/cloner** le projet que je viens de créer depuis ma VM avec la commande :

```
$git clone http://192.168.50.95/root/tp-devops.git
```

Je récupère donc le projet avec les mêmes **identifiants** que précédemment :

```
root@debian:~# git clone http://192.168.50.95/root/tp-devops.git
Clonage dans 'tp-devops'...
Username for 'http://192.168.50.95': root
Password for 'http://root@192.168.50.95':
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Réception d'objets: 100% (3/3), fait.
root@debian:~#
```

Configuration de la liste :

```
root@debian:~# git config --list
user.name=HOARAU Thomas
user.email=t.hoarau@rt-iut.re
root@debian:~#
```

Je constate que les informations contenues dans la liste sont les miennes.

Je vais maintenant ajouter le projet **GitLab** comme un dépôt distant.
Pour réaliser cela je vais suivre la partie “Add Your Files” indiqué sur le **GitLab**.

```
$cd /tp-devops
$git remote add origin http://192.168.50.95/root/tp-devops.git
$git branch -M master
$git push -uf origin master
```

Une fois ces commandes lancées j’ai ajouté le **projet GitLab** comme **dépôt distant**.

Maintenant pour **tester** le tout je vais créer un fichier contenant une **application** et un autre contenant les **dépendances**.

Fichier app.py :

```
  GNU nano 8.4                                app.py *
from flask import Flask
app = Flask(__name__)
@app.route("/")
def hello():
 return "Hello, World!", 200
if __name__ == "__main__":
 app.run(host="0.0.0.0", port=5000)
```

Fichier requirements.txt

```
  GNU nano 8.4                          requirements.txt *
flask
pytest
requests
```

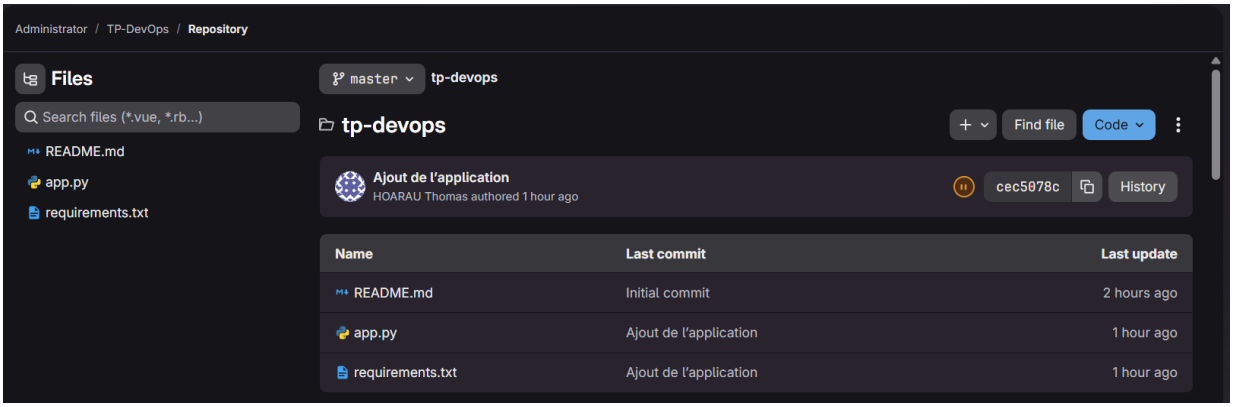
J'ajoute ensuite les **modifications** dans la **staging area**, puis je les déplace vers le **dépôt Git** en y ajoutant une description avec le commit.

```
root@debian:~/tp-devops# git add app.py
root@debian:~/tp-devops# git add requirements.txt
root@debian:~/tp-devops# git commit -m "Ajout de l'application"
[master cec5078] Ajout de l'application
 2 files changed, 11 insertions(+)
 create mode 100644 app.py
 create mode 100644 requirements.txt
root@debian:~/tp-devops#
```

Je vais ensuite **push** les **modifications** sur le **dépôt distant Gitlab** avec la commande :

```
root@debian:~/tp-devops# git push -u origin master
Username for 'http://192.168.50.95': root
Password for 'http://root@192.168.50.95':
Énumération des objets: 5, fait.
Décompte des objets: 100% (5/5), fait.
Compression des objets: 100% (3/3), fait.
Écriture des objets: 100% (4/4), 497 octets | 497.00 Kio/s, fait.
Total 4 (delta 0), réutilisés 0 (delta 0), réutilisés du paquet 0 (depuis 0)
remote:
remote: To create a merge request for master, visit:
remote:   http://192.168.50.95/root/tp-devops/-/merge_requests/new?merge_request
%5Bsource_branch%5D=master
remote:
To http://192.168.50.95/root/tp-devops.git
   bbe62d0..cec5078  master -> master
la branche 'master' est paramétrée pour suivre 'origin/master'.
root@debian:~/tp-devops#
```

Pour vérifier que les **modifications** ont bien été **push** sur le **Gitlab** il suffit de se rendre sur l'interface graphique du gitlab et de **consulter** le projet "tp-devops" :



Ici je vois bien que les fichiers précédemment créés sont présent sur la **branche master** du **dépôt distant**.

Questions :

10. Que signifie git remote add origin ?

Cette commande permet de faire le **lien** entre le **dépôt Git** de ma machine (repository local) et le **dépôt distant Gitlab**.

11. Pourquoi utiliser git push -u origin main au lieu de git push seul ?

Puisque lors du premier **push** il est nécessaire d'établir une **connexion permanente** entre les **branches**.

Ici je **crée un lien** entre la **branche master** de mon **dépôt local** et la **branche master** sur le **dépôt distant** (serveur origin).

12. Comment cloner un projet GitLab sur un nouvel ordinateur ?
Pour **cloner** le **projet** sur une autre machine il suffit d'installer Git sur cette machine et de **récupérer** le projet avec l'URL via la commande **git clone**.

II. CI/CD

Le **CI/CD** est un ensemble d'automates qui peuvent se déclencher après un **push** sur **Git.**

CI (Continuous Integration / Intégration Continue)

Dès que j'envoie un code, le serveur de CI va :

- **Build :** Compiler le code pour vérifier qu'il n'y a pas **d'erreurs** de syntaxe.
- **Test :** Lancer des **tests automatiques** pour s'assurer que tes modifs n'ont rien cassé ailleurs.

Le but est de détecter les bugs le plus tôt possible.

CD (Continuous Delivery ou Deployment / Déploiement Continu)

Une fois que les tests sont validé, le serveur de CD prend le relais :

- **Delivery :** Prépare une version prête à être **installée**.
- **Deployment :** Envoie automatiquement **l'application** sur le **serveur final** (site web, appli, etc.).

Le but est de déployer et mettre à jour un service simplement.

Ici on va prendre l'exemple de l'application précédente "Hello World" avec un **pipeline CI/CD GitLab** qui effectue les étapes suivantes :

- **1. Build** : Compilation de l'image Docker
- **2. Test** : Exécution d'un test basique, l'application Flask sert une page accessible sur http://localhost:5000/, qui affiche "Hello, World!"
- **3. Deploy** : Déploiement du conteneur sur un serveur

Étape 1 – Création de l'application

Création d'un Dockerfile pour l'application :

```
  GNU nano 8.4                          Dockerfile *
# Utilisation d'une image Python
FROM python:3.9-slim
# Définition du dossier de travail
WORKDIR /app
# Copie des fichiers et installation des dépendances
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
# Copie du code source
COPY . .
# Exposition du port et lancement de l'application
EXPOSE 5000
CMD ["python", "app.py"]
```

Ce **Dockerfile** va permettre par la suite de construire une **image Docker** de **l'application**.

Push le Dockerfile sur GitLab :

```
root@debian:~/tp-devops# git add Dockerfile
root@debian:~/tp-devops# git commit -m "Ajout du Dockerfile pour l'application"
[master b55ef07] Ajout du Dockerfile pour l'application
 1 file changed, 12 insertions(+)
 create mode 100644 Dockerfile
root@debian:~/tp-devops#
```

Étape 2 – Déploiement et vérification manuel

Questions :

13. Pouvez-vous lancer le dockerfile manuellement avec l'application ?

Pour lancer le **Dockerfile** de manière manuelle il faut **construire l'image**, pour cela j'utilise la commande :

```
$docker build -t app:1.0 .
```

Cette commande va permettre donc de **construire l'image** de mon **application** app:1.0 à partir du **DockerFile**.

Une fois **l'image construite** je peux **lancer le container** et effectuer un **mappage de port** sur le **port 5000** avec la commande :

```
$docker run -d -p 5000:5000 app:1.0
```

14. Que fait cette application ?

Cette application permet de :

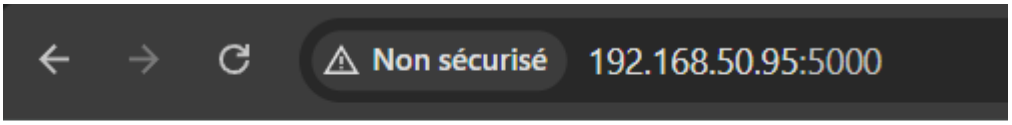
- **Créer un serveur Web :** Elle initialise un serveur web qui attend des connexions.
- **Réponds "Hello, World!"** : Dès qu'un utilisateur accède à cette page, l'application renvoie le texte "Hello, World!" avec un code de succès 200 (qui signifie que tout s'est bien passé).
- **Écoute sur le port 5000** : Elle est configurée pour recevoir les requêtes sur le port 5000 et accepte les connexions de n'importe quelle IP (0.0.0.0).

15. Pouvez-vous tester le fonctionnement de l'application ?

Une fois le container lancé avec la commande :

```
$docker run -d -p 5000:5000 app:1.0
```

On peut y accéder via **l'IP** de la machine et le **port 5000**.



Hello, World!

Étape 3 – En autonomie : Déploiement et vérification automatique

Pour cette dernière étape je vais mettre en place le **déploiement** et les **vérification automatiques** via **GitLab Runner**.

1. Installation GitLab Runner

Pour installer **GitLab Runner** je vais récupérer une **image Docker** pour ensuite **déployer** un **container** faisant tourner **GitLab Runner**.

Pour cela je lance la commande suivante :

```
$sudo docker run -d --name gitlab-runner --restart always \
  -v /srv/gitlab-runner/config:/etc/gitlab-runner \
  -v /var/run/docker.sock:/var/run/docker.sock \
  gitlab/gitlab-runner:latest
```

Cette commande permet de **lancer un container** avec **l'image de gitlab-runner** et un **mappage de volume** pour **autoriser** Gitlab Runner à **utiliser docker**.

```
root@debian:~/tp-devops# sudo docker run -d --name gitlab-runner --restart alway
s \
  -v /srv/gitlab-runner/config:/etc/gitlab-runner \
  -v /var/run/docker.sock:/var/run/docker.sock \
  gitlab/gitlab-runner:latest
Unable to find image 'gitlab/gitlab-runner:latest' locally
latest: Pulling from gitlab/gitlab-runner
a3629ac5b9f4: Pull complete
26ea538bff66: Pull complete
fad4846c3cc9: Pull complete
071471dce939: Pull complete
8673d8b79066: Pull complete
Digest: sha256:162b71b9771563969ca4efb602fa321ae5d62294cc40be077e7d408b3357b148
Status: Downloaded newer image for gitlab/gitlab-runner:latest
917465479e99f659fbe62031f6c8ba7526576c281d4f1b4f493736e16331ae9f
```

Maintenant que le **Gitlab Runner** est lancé sur ma machine il faut l'enregistrer auprès du **GitLab**.
En effet, le **conteneur** est lancé, mais il ne sait pas encore à quel **serveur GitLab** il doit **obéir**.
Il est donc nécessaire d'effectuer l'enregistrement, pour cela je procède de la façons suivante :

- Sur l'interface GitLab je me rends dans *Settings > CI/CD > Runners*.
- Je clique sur **New project runner** et récupère le **Token** d'enregistrement.

Token :
glrt-JYT6Nr1D-W5hh9ZuGbzU4G86MQpwOjEKdDozCnU6MQ8.01.1710smejw

Ensuite, dans le terminal je lance la configuration interactive :

```
$sudo docker exec -it gitlab-runner gitlab-runner register
```

Je rentre ensuite :

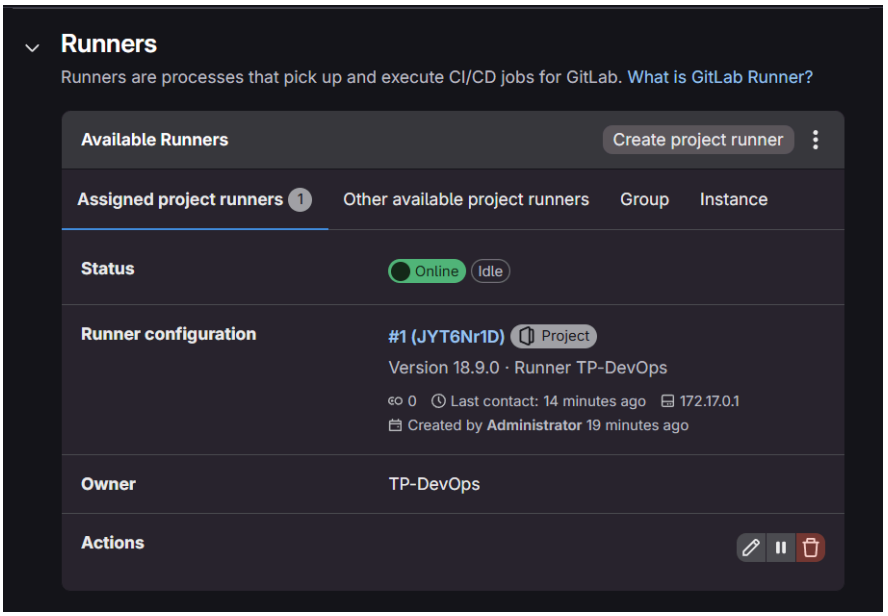
- L'URL : http://192.168.50.95/
- Token d'enregistrement
- Nom du Runner : TP-DevOps
- L'exécuteur : Docker
- L'image par défaut : python:3.9

```
root@debian:~/tp-devops# sudo docker exec -it gitlab-runner gitlab-runner regist
er
Runtime platform                                    arch=amd64 os=linux pid=29 r
evision=07e534ba version=18.9.0
Running in system-mode.

Enter the GitLab instance URL (for example, https://gitlab.com/):
http://192.168.50.95
Enter the registration token:
glrt-JYT6Nr1D-W5hh9ZuGbzU4G86MQpwOjEKdDozCnU6MQ8.01.1710smejw
Verifying runner... is valid                        correlation_id=01KJ7JTNDE93D
KCS90RT35A29P runner=JYT6Nr1D- runner_name=917465479e99
Enter a name for the runner. This is stored only in the local config.toml file:
[917465479e99]: TP-DevOps
Enter an executor: kubernetes, docker-autoscaler, instance, custom, ssh, virtual
box, docker+machine, shell, parallels, docker, docker-windows:
docker
Enter the default Docker image (for example, ruby:3.3):
python:3.9
Runner registered successfully. Feel free to start it, but if it's running alrea
dy the config should be automatically reloaded!

Configuration (with the authentication token) was saved in "/etc/gitlab-runner/c
onfig.toml"
root@debian:~/tp-devops#
```

Je peux ensuite **vérifier** que le **runner** est **actif** depuis l'interface Gitlab :



Je vais maintenant créer les **fichiers** nécessaires au fonctionnement du **CI/CD**.

Test_app.py

```
  GNU nano 8.4                                test_app.py *
import requests

def test_hello_world():
    # Note : Dans le pipeline, on testera l'app qui tourne
    response = requests.get("http://localhost:5000/")
    assert response.status_code == 200
    assert response.text == "Hello, World!"
```

Configuration de la **Pipeline CI/CD** avec le fichier .gitlab-ci.yml.

```
  GNU nano 8.4                          .gitlab-ci.yml *
# Définition des variables
variables:
 IMAGE_NAME: "$CI_REGISTRY_IMAGE/hello-world:$CI_COMMIT_REF_NAME"
stages:
 - build
 - test
 - deploy
before_script:
 - echo "Authentification au GitLab Container Registry"
 - echo $CI_REGISTRY_PASSWORD | docker login -u $CI_REGISTRY_USER --password-st>
# Étape 1 : Build de l'image Docker
```

***Le contenu du fichier .gitlab-ci.yml est disponible en annexe**

J'envoie le tout sur le **dépôt Gitlab** :

```
root@debian:~/tp-devops# git add .
root@debian:~/tp-devops# git commit -m "Ajout du Runner et Test de la pipeline C
I"
[master d6f2b45] Ajout du Runner et Test de la pipeline CI
 2 files changed, 45 insertions(+)
 create mode 100644 .gitlab-ci.yml
 create mode 100644 test_app.py
```

```
root@debian:~/tp-devops# git push origin master
Username for 'http://192.168.50.95': root
Password for 'http://root@192.168.50.95':
Énumération des objets: 5, fait.
Décompte des objets: 100% (5/5), fait.
Compression des objets: 100% (4/4), fait.
Écriture des objets: 100% (4/4), 940 octets | 42.00 Kio/s, fait.
Total 4 (delta 1), réutilisés 0 (delta 0), réutilisés du paquet 0 (depuis 0)
remote:
remote: To create a merge request for master, visit:
remote:   http://192.168.50.95/root/tp-devops/-/merge_requests/new?merge_request
%5Bsource_branch%5D=master
remote:
```

Remarque :
Pour les derniers tests l'IP de la machine n'est pas la même étant donné les problèmes rencontrés sur le stockage de la machine initialement utilisé, il était donc nécessaire de recommencer sur une machine ayant plus de ressource.

De plus, pour faire fonctionner le tout j'ai dû supprimer les containers qui tournaient sur le port 5000 pour le libérer et apporter certaines modifications au fichier .gitlab-ci.yml.

Questions :

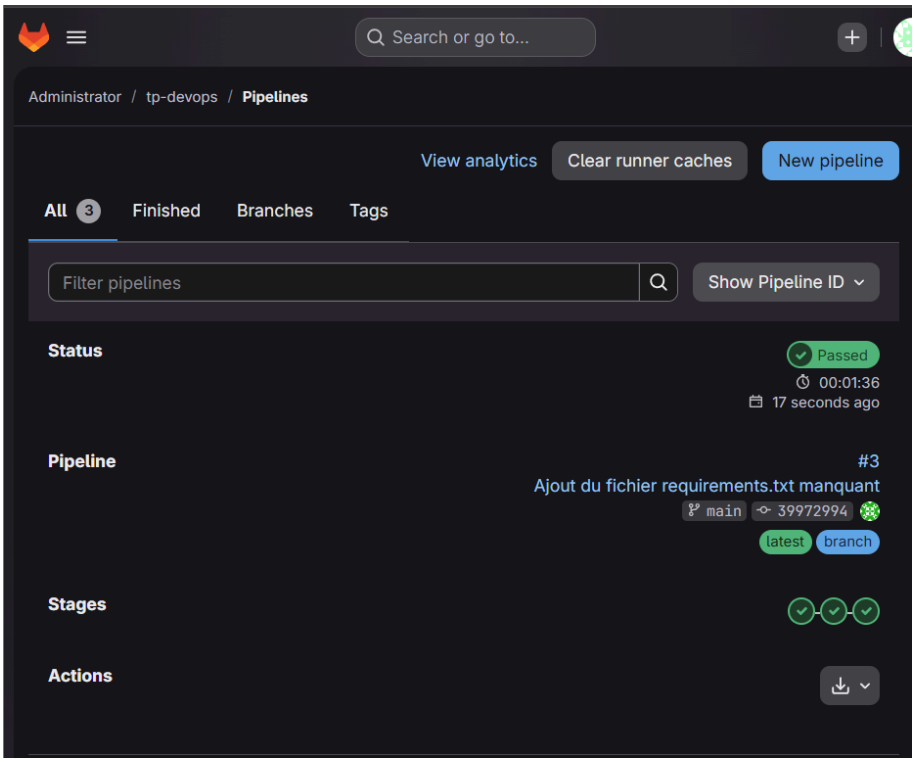
16. Pouvez-vous lancer une action git comme un git push et faire en sorte que cela lance automatiquement le pipeline ?
Oui grâce au **runner** et au fichier .gitlab-ci.yml à chaque **git push** la **pipeline CI/CD** est lancée **automatiquement** ce qui permet de gérer les **tests** et le **déploiement** de manière **automatique**.

17. Expliquer le fichier test_app.py.
Ce fichier est un **script de test** automatisé, il sert à vérifier que l'application Flask répond correctement.

18. Expliquer la pipeline .gitlab-ci.yml
C'est un **script** qui garantit que chaque modification de code est **automatiquement testée** et **mise en production**.
La **pipeline** découpe le projet en 3 partie :

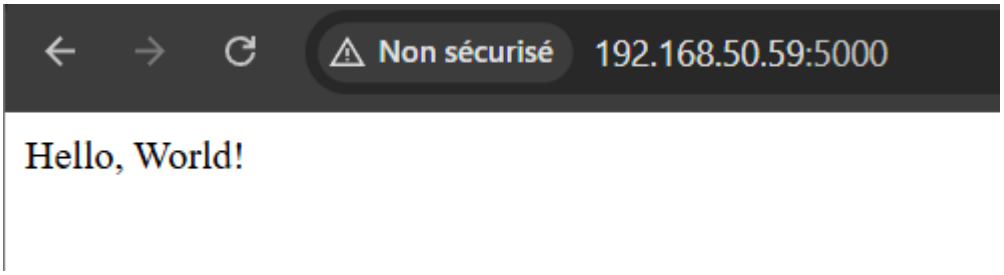
- **Build**: Le Runner utilise le **Dockerfile** pour créer une image Docker
- **Test** : Le **Runner** lance un **conteneur temporaire** pour tester les modifications apporté
- **Deploy** : Si les tests réussissent, le Runner va sur la VM, arrête l'ancienne version de l'application et lance le nouveau conteneur sur le port 5000.

19. Pouvez-vous vérifier que la pipeline CI/CD se lance correctement ?
Pour vérifier que la **pipeline CI/CD** que j'ai créer fonctionne, je me rends sur l'interface de **GitLab** et je clique sur *Build > Pipelines* dans le menu à gauche.



Ici toutes les étapes sont au **vert** (Build, Test , déploiement) ce qui signifie que la **pipeline CI/CD** se lance **correctement**.

20. L'application est bien mise à jour sur gitlab ?
Pour **vérifier** que **l'application** est bien **mise à jour**, je tape **l'IP** suivi du **port 5000** dans l'URL.



Ici on accède à la **page hello world**, l'application est donc **bien mise à jour**.

III. Conclusions

Ce TP m'a permis de mettre en place une chaîne de **DevOps** complète, allant de l'écriture du code jusqu'à son déploiement automatique.

Points clés retenus :

- **Conteneurisation avec Docker** : Déploiement de plusieurs containers.
- **Intégration Continue (CI)** : Configuration du fichier .gitlab-ci.yml
- **Déploiement Continu (CD)** : Configuration et déploiement du Gitlab Runner.

IV. Annexe

Contenu fichiers .gitlab-ci.yml :

```
variables:
  IMAGE_NAME: "tp-devops-app:latest"

# L'image par défaut pour les jobs qui utilisent Docker
image: docker:latest

stages:
  - build
  - test
  - deploy

# Étape 1 : Construction de l'image Docker
build_job:
  stage: build
  script:
    - docker build -t $IMAGE_NAME .
  only:
    - master

# Étape 2 : Tests unitaires et fonctionnels
test_job:
  stage: test
```

```
  image: python:3.9
  script:
    # Installation des dépendances pour le test
    - pip install flask requests pytest
    # Lancement de l'application en arrière-plan (&)
    - python app.py &
    # On laisse 5 secondes à Flask pour démarrer proprement
    - sleep 5
    # Exécution du script de test
    - pytest test_app.py
  only:
    - master

# Étape 3 : Déploiement sur la VM
deploy_job:
  stage: deploy
  script:
    # On arrête l'ancien conteneur s'il existe pour éviter les
conflits de port
    - docker stop hello-container || true
    - docker rm hello-container || true
    # On lance le nouveau conteneur sur le port 5000
    - docker run -d -p 5000:5000 --name hello-container
$IMAGE_NAME
  only:
    - master
```